TD Noyaux Temps-réel

1 Tâches

La tâche init est créée dans le main (non-représenté ici).

```c
#include <stdio.h>
#include <FreeRTOS.h>

#define STACK_DEPTH 256
#define BASE_PRIO 1

int x = 0;
void codeTache(void* num) {
    int j;

    for (j = 0 ; j < 10 ; j++) {
        x++;
        printf("tache %d, j = %d, x = %d\r\n", (int)num, j, x);
        vTaskDelay(1);
    }
    vTaskDelete(NULL);
}

/* Tâche de priorité BASE_PRIO */
void init(void* param) {
    int i;
    TaskHandle_t xHandle = NULL;

    for (i = 0 ; i < 4 ; i++) {
        xTaskCreate(codeTache, "", STACK_SIZE, (void *) i, BASE_PRIO+i+1, &xHandle);
        printf("Creation de la tache %d\r\n", i);
    }

    vTaskDelete(NULL);
}
```

On supposera que la durée d'exécution de la fonction printf est négligeable et qu'elle ne provoque pas de changement d'état des tâches.

1. Donner les 12 premières lignes qui s'affichent à l'écran.
2. La fonction codeTache est-elle réentrante ?

2 Sémaphores

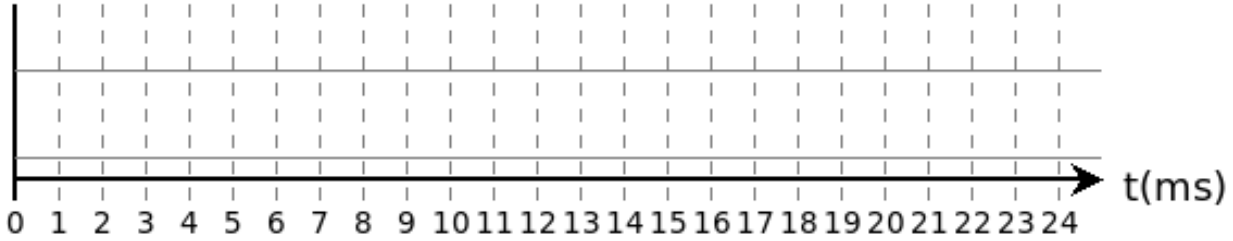
Soit le code suivant :

```
void task1(void * p) {
    for(;;) {
        process1();    // dure 4ms
    }
}

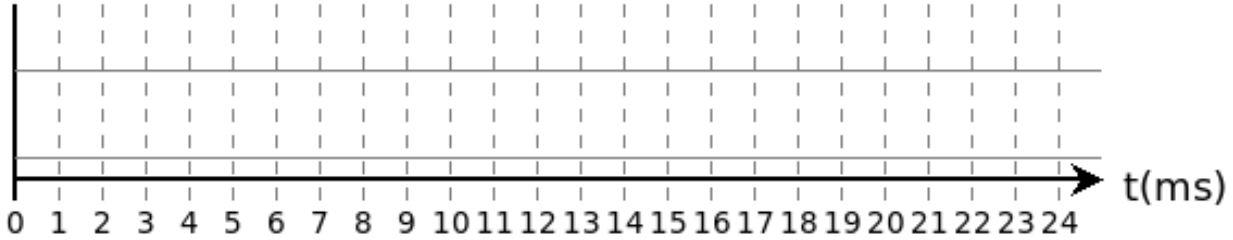
void task2(void * p) {
    for(;;) {
        process2();    // dure 2ms
    }
}
```

- Les tâches sont crées dans la fonction `main()` et le *scheduler* de FreeRTOS est démarré. On ne s'intéresse pas à cette partie dans cet exercice.
- La fonction `process1()` dure **4ms** et doit être appelé une fois toutes les **8ms**.
- La fonction `process2()` dure **2ms** et doit être appelé une fois toutes les **10ms**.
- `process2()` est prioritaire sur `process1()`.
- task2 est donc de priorité 2 et task1 de priorité 1.

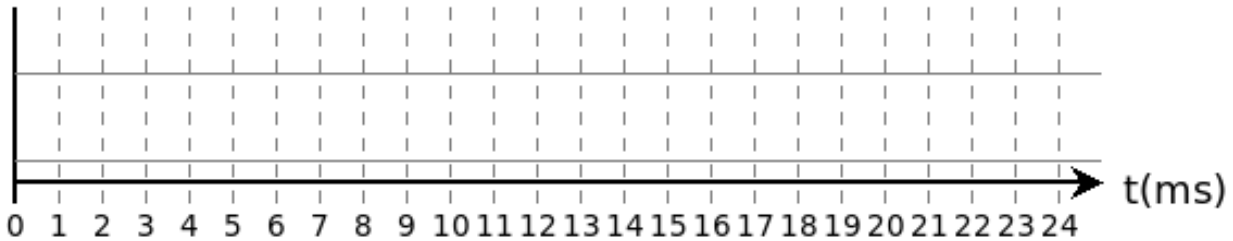
1. À l'aide de la fonction vTaskDelay, proposez une modification pour que les timings soient respectés.
2. Tracer le chronogramme. Est-ce satisfaisant ?



3. À l'aide d'une troisième tâche taskSync et de sémaphores binaires, proposez une meilleure solution.
4. Quelle doit être la priorité de taskSync ?
5. Tracer le chronogramme.



6. Imaginons maintenant que les fonctions process1() et process2() se partagent une ressources critique. Elles ne doivent donc pas se préempter mutuellement.
7. Proposer une solution à l'aide d'un sémaphore mutex.
8. Tracer le chronogramme.



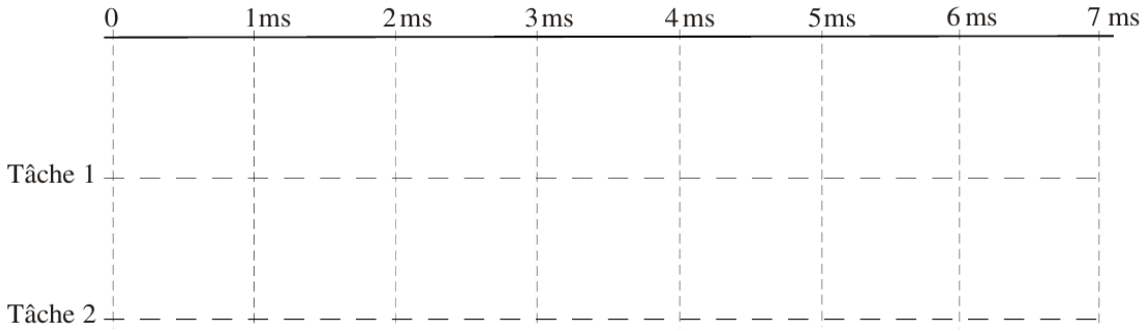
Les tâches 1 et 2 sont crées dans le main et le scheduler est démarré.

```
void tache1 (void) {
    while(1) {
        xSemaphoreTake(S, portMAX_DELAY);
        traitement();
    }
}
```

```
void tache2(void) {
    while(1) {
        traitement();
        xSemaphoreGive(S);
        traitement();
        vTaskDelay(2);
    }
}
```

La fonction `traitement()` dure 500 µs. On négligera la durée de traitement des primitives systèmes. Le sémaphore binaire S est créé dans le main et initialisé à 0 (sémaphore vide).

La priorité de la tache 1 est de 11, et celle de la tâche 2 est de 10. En considérant que les deux tâches sont prêtes à l'instant 0, donner l'état des deux tâches sur le dessin suivant :



Soit le code suivant. Qu'est-ce qui s'affiche à l'écran ?

```
SemaphoreHandle_t sem1, sem2;
TaskHandle_t t1, t2;

int main (void) {
    sem1 = xSemaphoreCreateBinary();
    sem2 = xSemaphoreCreateBinary();
    xSemaphoreGive(sem1);
    xSemaphoreGive(sem2);

    xTaskCreate(tache1, "t1", 1000, NULL, 7, &t1);
    xTaskCreate(tache2, "t2", 1000, NULL, 5, &t2);

    vTaskStartScheduler();

    return 0;
}
```

```
void tache1 (void * pvParameters) {
    printf("t1: prend sem1\n");
    xSemaphoreTake(sem1, portMAX_DELAY);
    printf("t1: sem1 pris\n");

    vTaskDelay(1);

    printf("t1: prend sem2\n");
    xSemaphoreTake(sem2, portMAX_DELAY);
    printf("t1: sem2 pris\n");

    vTaskDelay(1);

    printf("t1: donne sem1\n");
    xSemaphoreGive(sem1);
    printf("t1: sem1 rendu\n");

    vTaskDelay(1);

    printf("t1: donne sem2\n");
    xSemaphoreGive(sem2);
    printf("t1: sem1 rendu\n");

    vTaskDelete(NULL);
}
```

```
void tache2 (void * pvParameters) {
    printf("t2: prend sem2\n");
    xSemaphoreTake(sem2, portMAX_DELAY);
    printf("t2: sem2 pris\n");

    vTaskDelay(1);

    printf("t2: prend sem1\n");
    xSemaphoreTake(sem1, portMAX_DELAY);
    printf("t2: sem1 pris\n");

    vTaskDelay(1);

    printf("t2: donne sem2\n");
    xSemaphoreGive(sem2);
    printf("t2: sem2 rendu\n");

    vTaskDelay(1);

    printf("t2: donne sem1\n");
    xSemaphoreGive(sem1);
    printf("t2: sem1 rendu\n");

    vTaskDelete(NULL);
}
```

3 Boîtes aux lettres

La tâche liée à la fonction `init()` est crée et le *scheduleur* est démarré. Qu'est-ce qui s'affiche à l'écran ?

```
#define BASE_PRIO 5
#define STACK_SIZE 1000
#define NBR_MAX 3
#define LONG_MAX 100

QueueHandle_t maBoite;

void tache2 (void* p) {
    char messBuf[LONG_MAX];
    int i;

    printf("t2: alive\r\n");

    for (i = 0 ; i < 5 ; i++) {
        xQueueReceive(maBoite, (void *)messBuf, portMAX_DELAY);
        printf("t2: R %s\r\n", messBuf);
    }

    printf("t2 out\r\n");

    vTaskDelete(NULL);
}

void tache1(void* p) {
    char msg[] = "msg 00";
    int i;

    printf("t1: alive\r\n");

    maBoite = xQueueCreate(NBR_MAX, LONG_MAX);

    printf("t1: created Queue\r\n");

    for (i = 0 ; i < 5 ; i++) {
        sprintf(msg, "msg %02d", i);
        printf("t1: S %s\r\n", msg);
        xQueueSend(maBoite, (const void *) msg, portMAX_DELAY);
    }

    printf("t1 out\r\n");

    vTaskDelete(NULL);
}

void init(void* p) {
    TaskHandle_t xHandle1 = NULL;
    TaskHandle_t xHandle2 = NULL;
    xTaskCreate(tache1, "Tache1", STACK_SIZE, NULL, BASE_PRIO-1, &xHandle1);
    xTaskCreate(tache2, "Tache2", STACK_SIZE, NULL, BASE_PRIO-2, &xHandle2);

    vTaskDelete(NULL);
}
```

4 Étude d'un Shell

4.1 Découverte

1. Téléchargez le projet suivant : `https://github.com/lfiack/rtos_td_shell`
2. Importez le projet dans STM32CubeIDE, compilez.
3. Lancez TeraTerm, Putty, ou un autre émulateur de terminal (ou en dernier recours celui intégré à CubeIDE).
4. Lancez le programme. Dans le terminal, afficher l'aide du shell en appuyant sur 'h'.
5. Depuis le terminal, appelez la fonction inutile.

4.2 Le shell pour l'utilisateur

6. Modifiez la fonction inutile pour afficher les arguments passés par l'utilisateur.
7. Ajoutez une fonction permettant d'effectuer une addition entre deux paramètres.

4.3 Analyse du code

8. Listez les fonctions accessibles à l'utilisateur. Listez les fonctions masquées.
9. Comment choisit-on la liaison série sur laquelle écrire ?
10. Listez les variables et les structures statiques. Que signifie le mot-clé `static` ?
11. Dans quelle fonction les caractères sont-ils lus ?
12. Comment fonctionne la suppression de caractères ?
13. Que se passe-t-il lorsque l'on appuye sur Entrée ?
14. À quelle fonction est passée la chaine de caractère ?
15. Dans cette fonction, où la fonction est-t-elle appelée ?
16. Comment les fonctions sont-elles ajoutées à la liste ?
17. En analysant le code des deux fonctions précédentes, expliquez le fonctionnement du shell.
18. Expliquez les mécanismes qui mènent à l'exécution de la fonction.

4.4 Améliorations

19. Qu'est-ce qui nous freine pour intégrer le shell à une tâche FreeRTOS ?
20. Proposer une solution.
21. Peut-on utiliser le shell sur plusieurs liaisons séries ? Simultanément ?
22. Peut-on facilement porter le shell sur un microcontrôleur d'un autre constructeur ?
23. Proposez des solutions pour rendre le shell plus portable.

5 Étude de la HAL

5.1 Initialisation de l'UART avec la HAL

1. Ouvrez le projet du TD Shell
2. Activez les interruptions de l'USART1
3. Dans le fichier `main.c`, repérez la fonction `MX_USART1_UART_Init()`
4. Allez voir le code de cette fonction
5. Quel est le paramètre passé à la fonction `HAL_UART_Init()` ? Pourquoi utiliser un pointeur ?
6. Quels sont les membres de la structure ?
7. Allez voir le code de la fonction `HAL_UART_Init()`
8. Où est définie `HAL_UART_MspInit()` ?
9. Que fait la fonction `UART_SetConfig()` ?
10. Quel est le rôle de gState ? Quel est sa valeur au retour de la fonction `HAL_UART_Init()` ?

5.2 Transmission d'une chaine sans interruption

11. Que se passe-t-il si la HAL n'est pas dans l'état `HAL_UART_STATE_READY` ?
12. Peut-on utiliser un autre UART pendant ce temps-là ?
13. Expliquez le rôle de chacun des paramètres suivants :
    (a) pData
    (b) Size
    (c) Timeout
14. Peut-on appeler la fonction `HAL_UART_Transmit()` dans deux fonctions différentes en même temps ?
15. Est-ce que le code est réentrant ?

5.3 Réception d'une chaine avec interruptions

16. Avant de regarder le code, devinez le fonctionnement de la fonction `HAL_UART_Receive_IT`
17. Listez les fonctions utilisée par la réception
18. Où et par qui est appelée la fonction `UART_RxISR_8BIT()` ?
19. Quand la fonction `HAL_UART_RxCpltCallback()` est-elle appelée ?
20. Après analyse, vérifiez si votre supposition était correcte

6 Amélioration du Shell

Le projet propre est disponible ici : `https://github.com/lfiack/rtos_td_shell`

6.1 Ajout d'une structure

1. Listez les variables statiques
2. Dans le fichier `shell.h`, créez une structure nommée `h_shell_t`. Les membres de la structure correspondent aux variables statiques.
3. Modifiez toutes les fonctions du shell qui manipulent les variables statiques pour qu'elles prennent comme premier paramètre un pointeur sur la structure `h_shell_t`
4. Modifiez le contenu des fonctions pour utiliser les membres de la structure à la place des variables statiques
5. Créez une instance statique de cette structure dans le main. Modifiez l'appel aux fonctions dans le main pour y ajouter la structure.
6. Testez vos modifications.

6.2 Driver UART

7. Dans le fichier `shell.h`, ajoutez la ligne suivante :

   ```
   typedef uint8_t (* drv_shell_transmit_t)(char * pData, uint16_t size);
   ```

8. Qu'est-ce que ça signifie ?
9. Faites de même pour la fonction `drv_shell_receive_t`
10. Créez la structure suivante :

    ```
    typedef struct drv_shell_struct
    {
        drv_shell_transmit_t drv_shell_transmit;
        drv_shell_receive_t drv_shell_receive;
    } drv_shell_t;
    ```

11. Ajoutez une instance de cette nouvelle structure dans la structure `h_shell_t`
12. Créez deux fichiers `drv_uart.c` et `drv_uart.h`.
13. Dans ces fichiers, créez deux fonctions :

    ```
    uint8_t drv_uart_receive(char * pData, uint16_t size)
    uint8_t drv_uart_transmit(char * pData, uint16_t size)
    ```

14. Écrire le code de ces fonctions. Inspirez vous des fonctions `uart_read()` et `uart_write(char * s, uint16_t size)` dans `shell.c`.
15. Dans `shell.c`, modifiez le code pour utiliser ces nouvelles fonctions à la place des anciennes.
16. Modifiez le `main()` pour passer vos deux fonctions de driver à la structure du shell.

6.3 Intégration à FreeRTOS

17. Activez l'OS et démarrez le scheduler (`vTaskStartScheduler()`).
18. Créez une tâche dans laquelle vous appellerez `shell_run()`.
19. Modifiez le driver UART pour fonctionner avec interruptions. Utilisez une notification pour la synchronisation.
20. Testez.